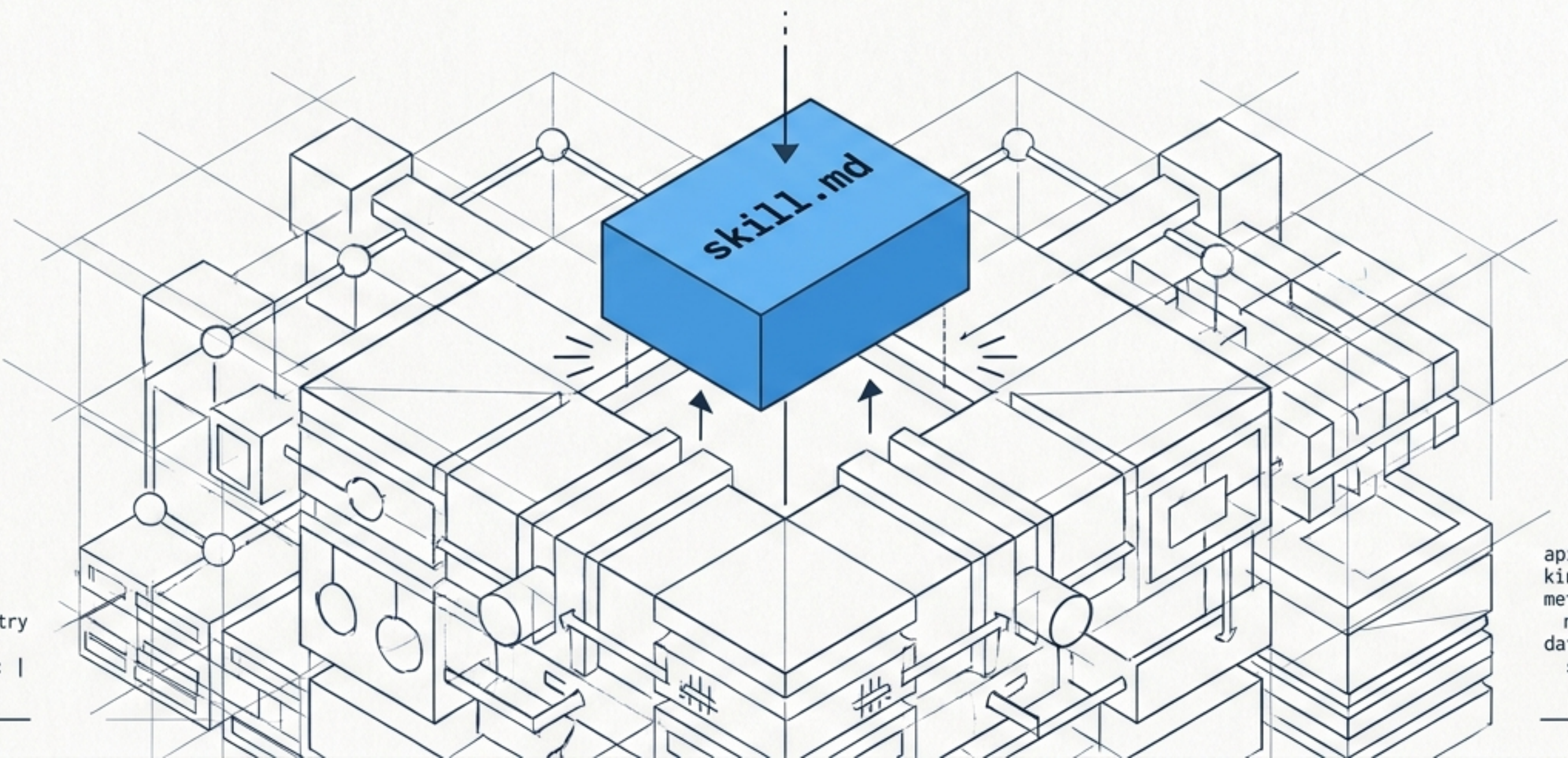


```
apiVersion: v1
kind: ConfigMap
metadata:
  name: skill-registry
data:
  skill_definition: |
  ...
```

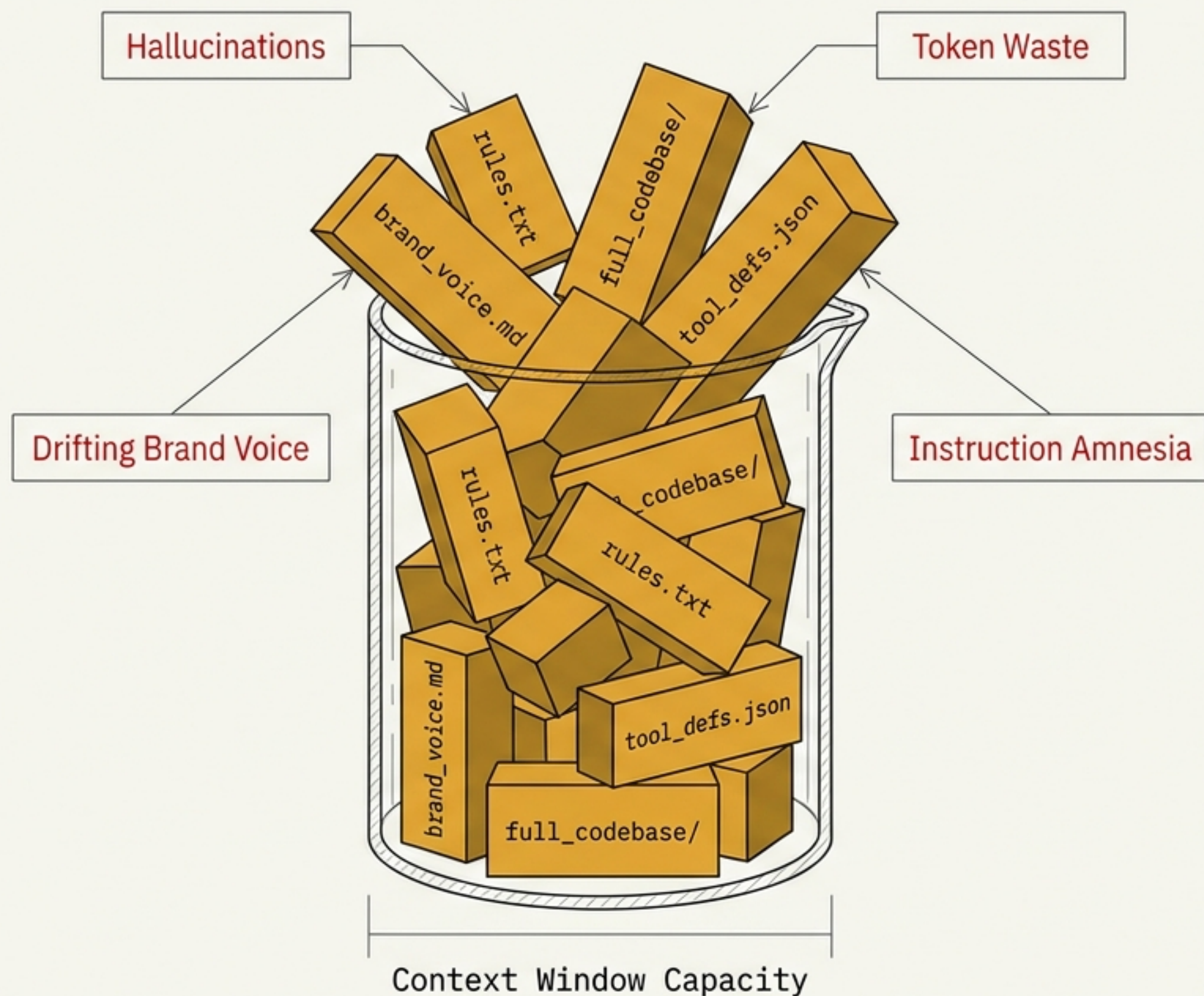
Escaping Context Bloat

A technical blueprint for **Agent Skills**, **Progressive Disclosure**, and **modular AI architectures**.



```
apiVersion: v1
kind: ConfigMap
metadata:
  name: skill-registry
data:
  skill_definition: |
  ...
```

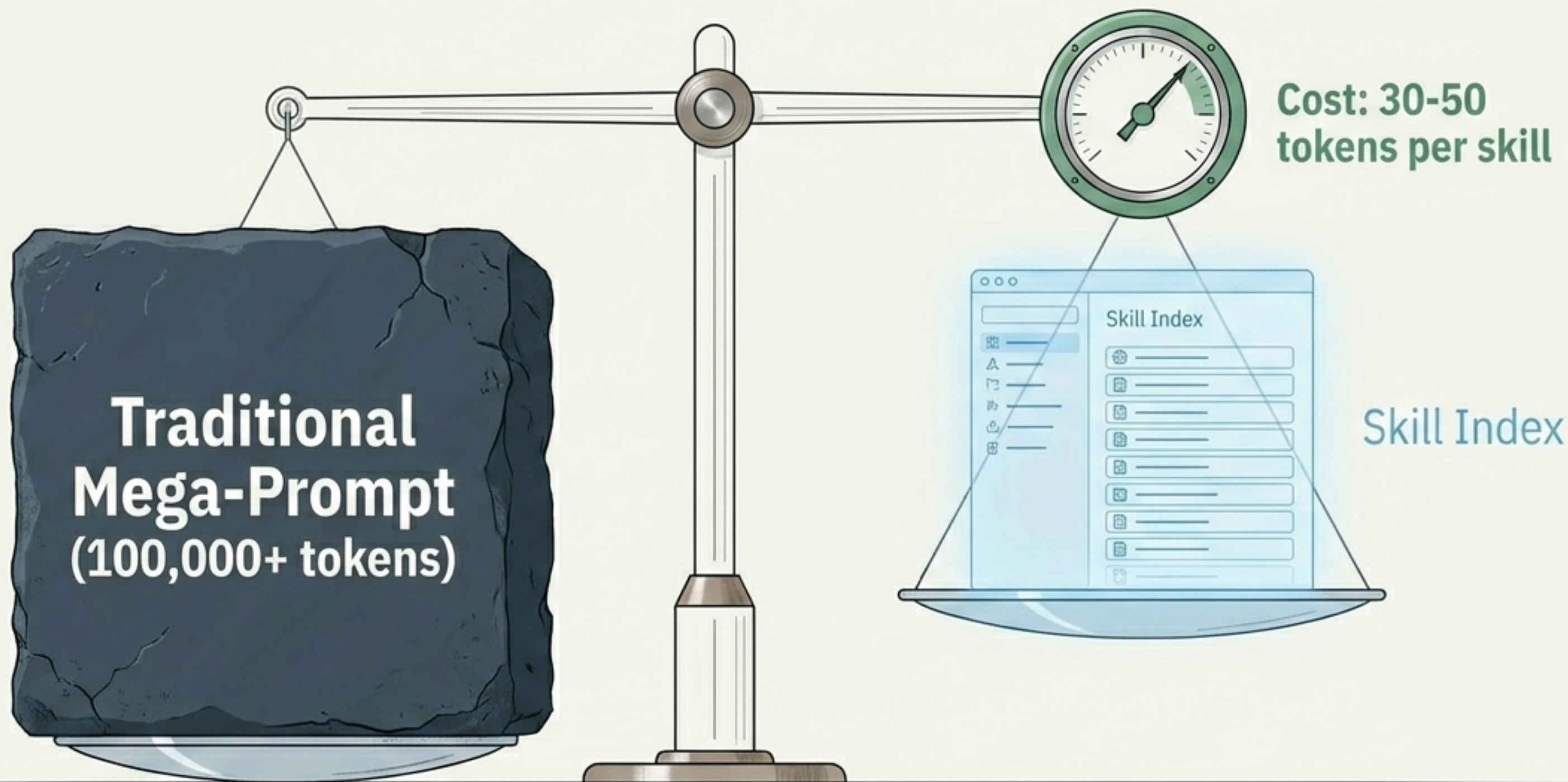
```
apiVersion: v1
kind: ConfigMap
metadata:
  name: skill-registry
data:
  skill_definition: |
  ...
```

The Wall We Hit

As applications scale, placing entire guides, codebases, and tool definitions into a single massive prompt hits a wall. Models perform worse as instructions get longer, forgetting rules and wasting expensive tokens.

We cannot fit the entire codebase into the context window.

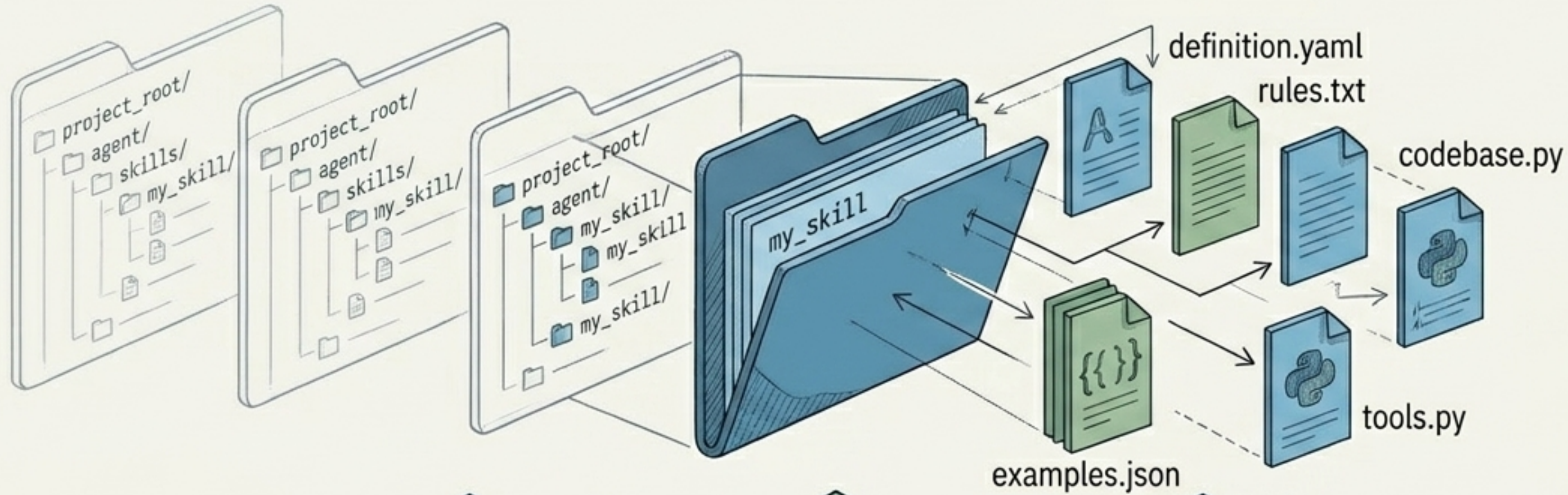


The Paradigm Shift

Instead of carrying every instruction all the time, the agent keeps a lightweight catalog of capabilities in context. It **pays the token cost only for what** it dynamically loads.

What is an Agent Skill?

A skill is an organized folder that packages **narrow, deep expertise**. It acts as a reusable **playbook** that an agent can **automatically discover and invoke** when relevant.



Portable

Works seamlessly across environments (Claude Code, API, Copilot CLI, Claude.ai).

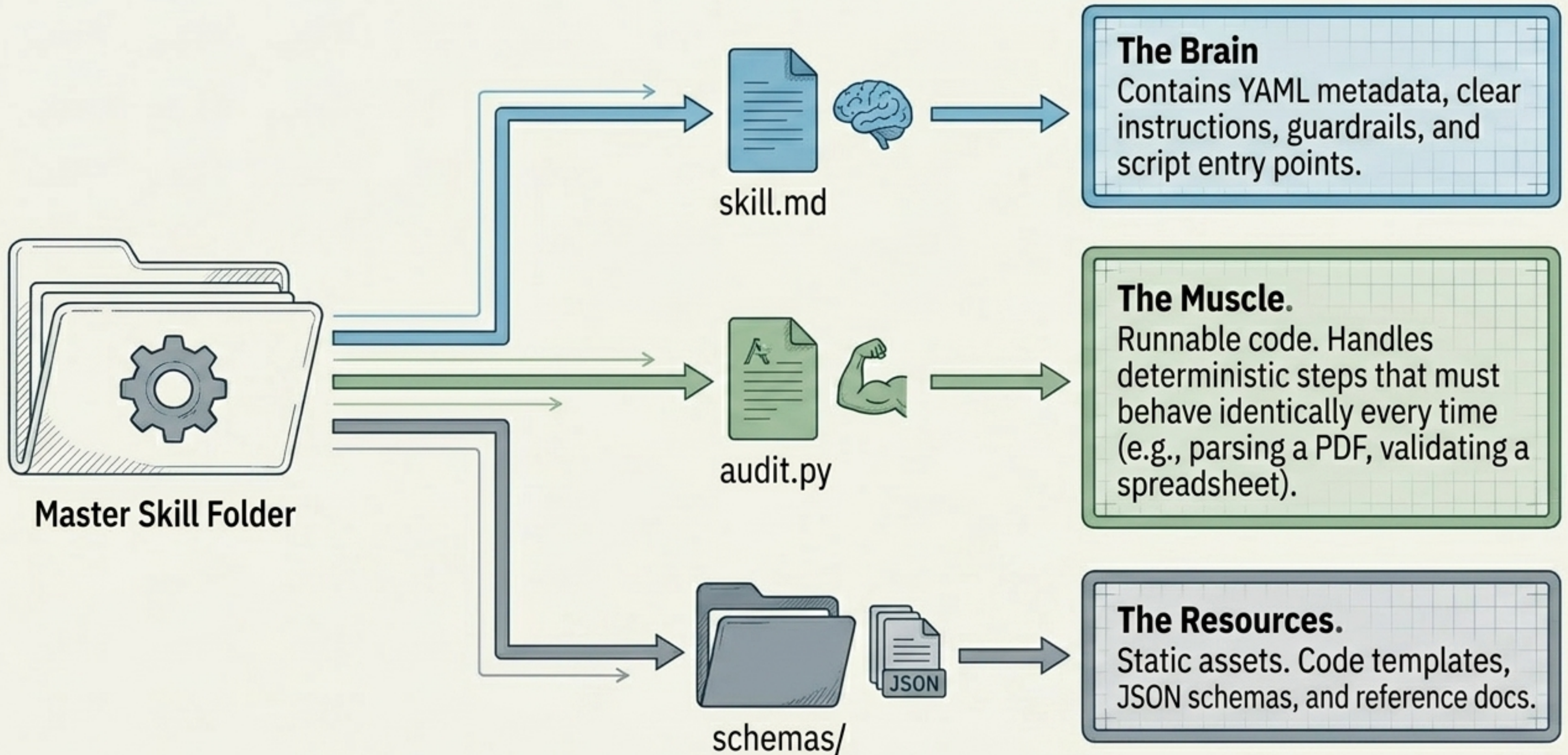
Versioned

Managed on disk in git repositories (e.g., .github/skills/, agent/skills/).

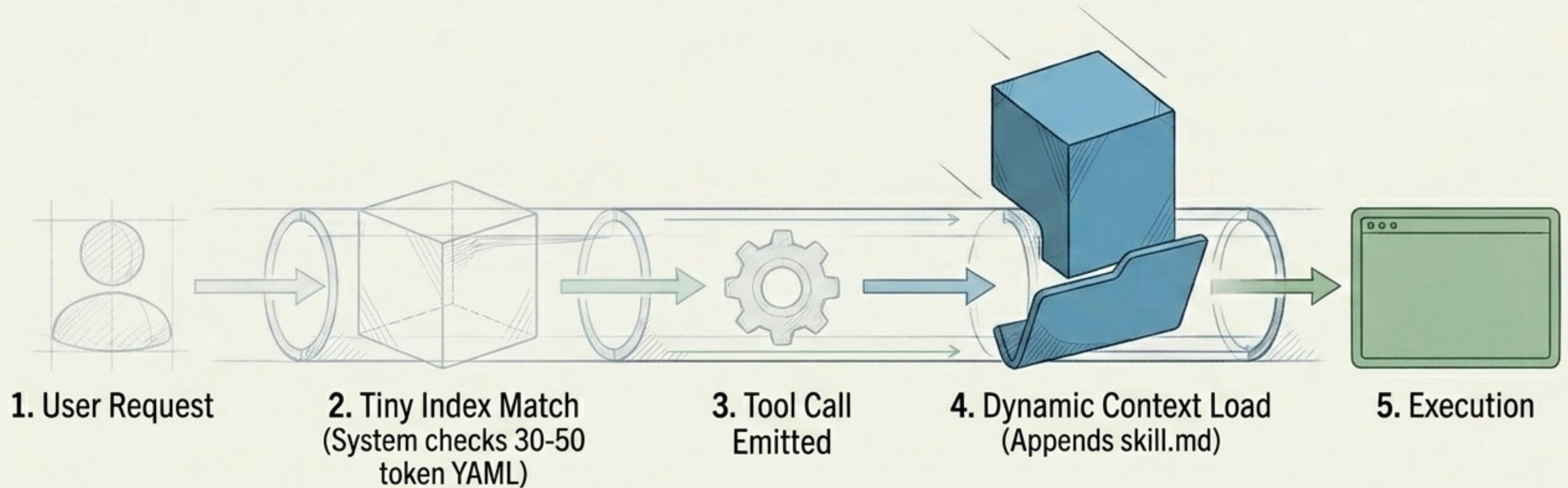
Collocated

Distributed directly alongside project source code.

Anatomy of a Skill

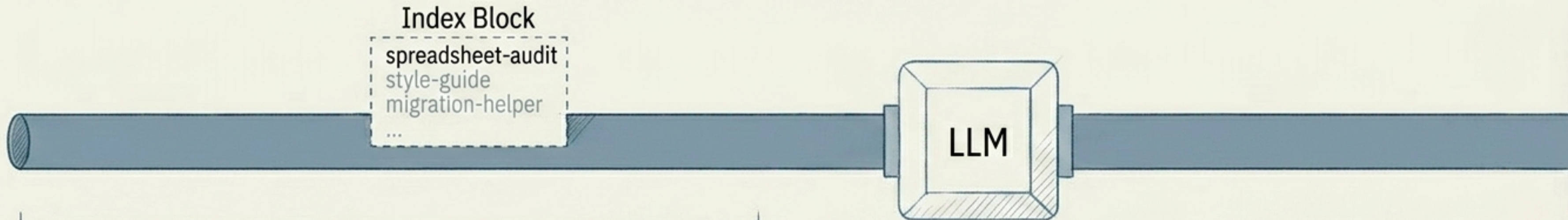
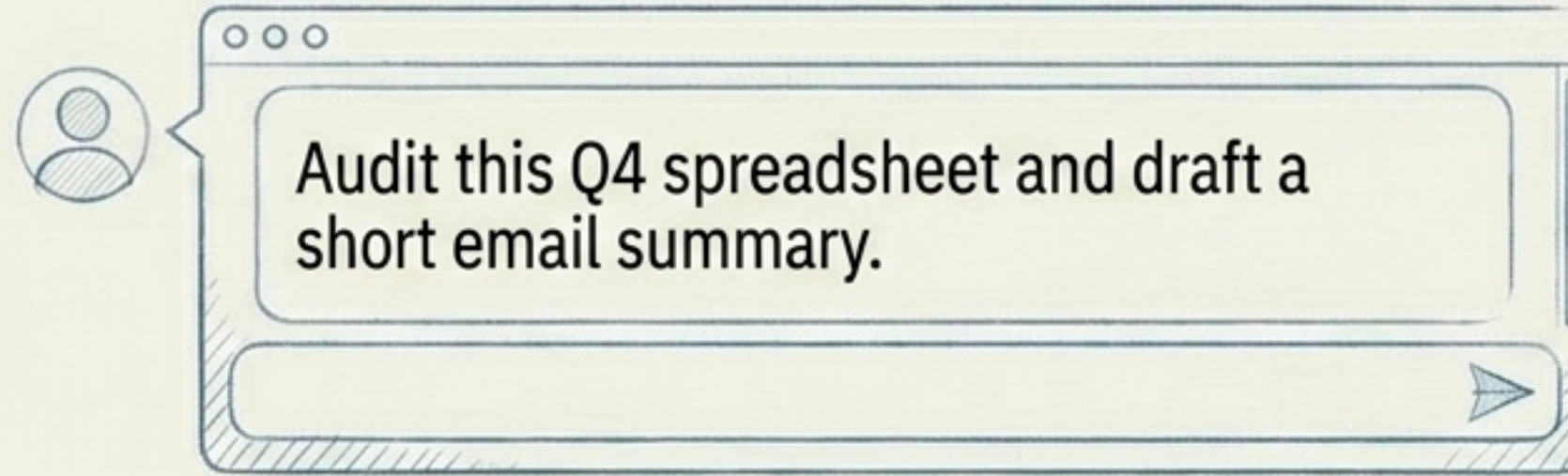


The Engine of Progressive Disclosure



Skills operate on **progressive disclosure**. Like a well-organized manual, the model starts with just a table of contents. It emits a tool call to request a specific chapter only when the user's prompt matches the skill's YAML description.

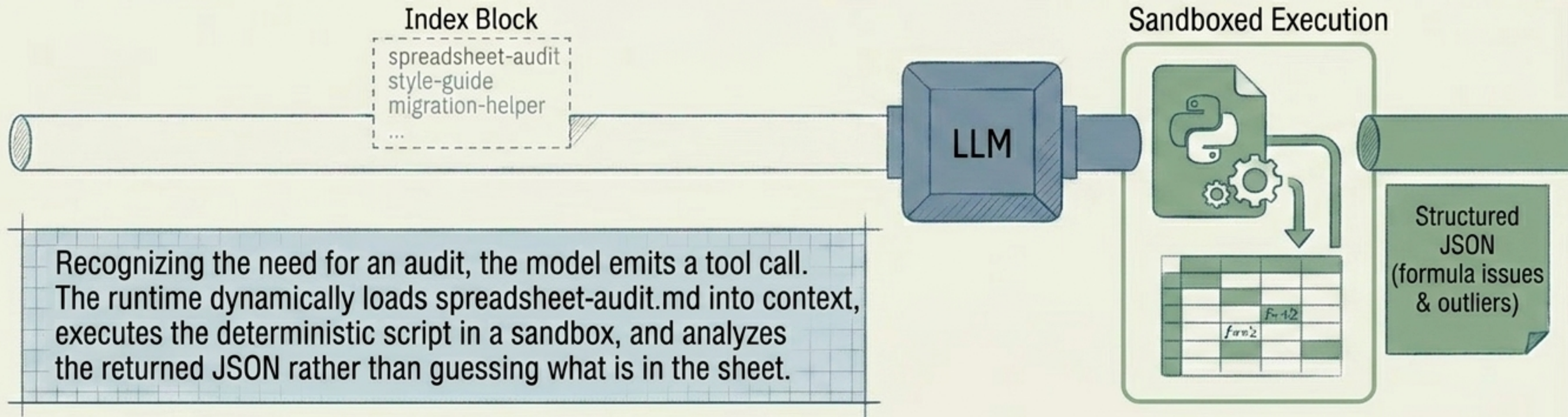
Workflow Map - Phase 1: The Request



The agent runtime appends only the lightweight skill **index** to the user request. The model sees the names and descriptions, but none of the actual instructions. Context weight remains near zero.

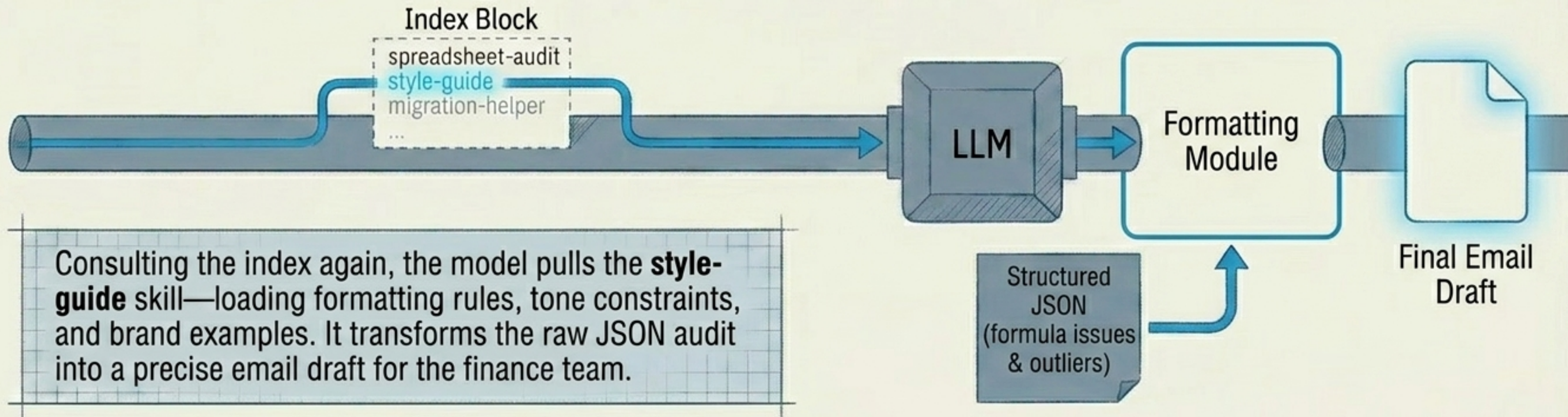
Workflow Map - Phase 2: Dynamic Execution

```
>_request_skill(spreadsheet-audit.md)
```



Workflow Map - Phase 3: Skill Chaining

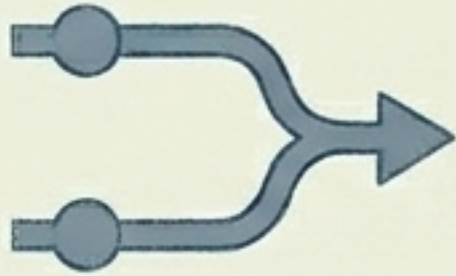
```
>_request_skill(style-guide.md)
```



Unlimited Domain Expertise

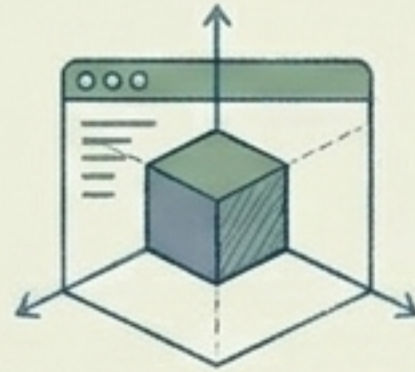
Skills are open standards (distributable via tools like `skills.sh`). They can package any hyper-specific workflow, from teaching an agent Tiny Shader Language to automating pull requests based on repository templates.

Make Contribution (DevOps)



-  Checks guidelines
-  Applies templates
-  Generates branches
-  Opens PRs

HTML GUI & 3JS (Frontend)



-  Resize listeners
-  Animation loops
-  Vite setups
-  Tiny Shader Language

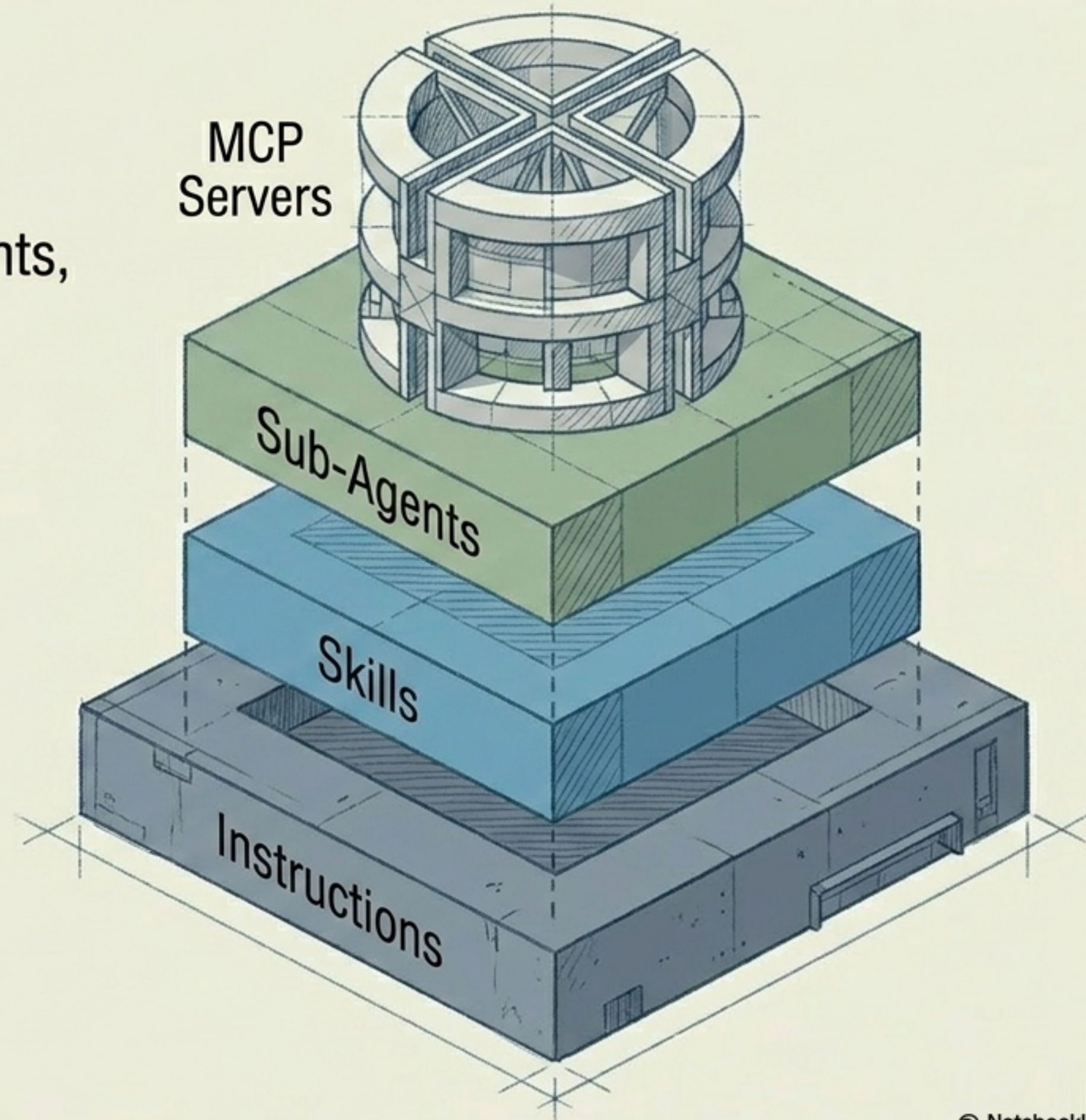
Security Onboarding (Team Ops)



-  Team coding standards
-  PR security best practices
-  Vulnerability checks

The AI Primitives Ecosystem

Skills do not exist in a vacuum. To build scalable agentic architectures, we must harmonize skills with Instructions, Sub-Agents, and the Model Context Protocol (MCP).



AI Primitives Diagnostics Matrix

	Instructions (claude.md)	Agent Skills (skill.md)	Sub-Agents	MCP Servers
Scope	Global/Project	Portable/Modular	Role-based	Universal Integration
Function	Foundational context (tech stack, conventions)	Deep procedural expertise (how to do a task)	Specialized workers with distinct context windows	Connects to external data sources (GitHub, Linear, Postgres)
Trigger	Always on	Lazily loaded	Assigned to tasks	Data retrieval

Instructions provide the baseline, **Sub-agents** provide the persona, MCP provides the raw data, and **Skills** provide the exact methodology.

Working in Harmony

Sub-Agent:
UI Reviewer

Skills:
Component
Patterns

MCP Server:
Linear Tickets

Instructions
(claude.md: We use Next.js)

MCP connects the data, Sub-agents specialize in their roles, your foundation files set the global codebase rules, and **Skills** bring the localized expertise.

A UI Sub-Agent can use a design system skill, while a Developer Sub-Agent uses a component skill—both accessing the exact same portable folder.



The Engineering ROI

- **Zero Prompt Surgery:** Update a script once; the behavior improves globally across all teammates.
- **Eliminate Bloat:** Keep context windows lightweight, reducing token spend and hallucination risk.
- **Portable Intelligence:** Distribute expert workflows effortlessly via git.

Agent skills replace brittle mega-prompts with scalable, versioned capabilities. They don't just teach the model what to do—they provide the exact tools to execute it perfectly every time.